

Haskell.py Teacher's Edition

A Complete Course in Idiomatic Functional Python

nine chapters, 41 homework problems, 10 compare-and-contrast apps

J. L. Clements III

September 18, 2026

How to Take This Course

This book teaches `haskell.py` – `prelude.py`’s production sibling. Where `prelude.py` reimplements every tool from scratch for pedagogy, `haskell.py` WRAPS the standard library wherever it already has the tool (`scanl` is `itertools.accumulate`, `memo` is `functools.cache`) – one vocabulary, zero reimplementation. Failure is a value throughout: a true NOTHING sentinel keeps `None` legal as everyday data, parsers fail with `FAIL`, and `Either` carries `WHY`.

The whole library is one listing, up front, in Chapter 1 – read it once, top to bottom, the way you would read any short source file. The nine chapters after it are not a second pass over the same material: each opens with why its slice of the library exists and how its pieces fit together, then goes straight to **Homework** – drills cement each tool, applies combine them, and challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in; `python3 -m doctest` is the referee.

Contents

How to Take This Course	iii
I The Nine Chapters	1
1 haskell.py, Complete	3
2 Point-Free Foundations	9
2.1 Homework	9
3 Folds, Scans and Unfolds	11
3.1 Homework	11
4 The List Gaps	13
4.1 Homework	13
5 Maybe, Done Right	15
5.1 Homework	15
6 Either: Failure With a Why	17
6.1 Homework	17
7 do-Notation	19
7.1 Homework	19
8 Parser Combinators	21
8.1 Homework	21
9 Monoids at Work	23
9.1 Homework	23
10 Capstone Labs	25
10.1 Homework	25
II Compare and Contrast: Imperative vs Functional	27
About this part	29
11 calculator	31
12 gradebook	33
13 layers	35

14	leaderboard	37
15	logtrriage	39
16	orderbatch	41
17	orgchart	43
18	payoff	45
19	sensoralert	47
20	wordfreq	49
III Appendices		51
A	haskell.py, Complete	53

Part I

The Nine Chapters

Chapter 1

haskell.py, Complete

Every name this book teaches, in one file, in the order it is defined – docstrings and comments stripped to the section headers, which are the only thing worth keeping as a wayfinding aid in a listing this short. Read it once before the first chapter; the chapters that follow assume you have.

```
1 from functools import reduce as _reduce, partial, cache as memo, cmp_to_key
2 from itertools import accumulate as _acc, islice as _islice, pairwise as _pairwise
3 from itertools import chain, count, cycle, repeat
4 from operator import add, sub, mul, not_
5 from math import gcd, lcm, hypot, isqrt, prod as product
6 from collections import Counter, defaultdict, deque
7 from heapq import heappush, heappop, merge as _hmerge
8 import re as _re
9 # 1. functions
10 identity = lambda x: x
11 const = lambda x: lambda _: x
12 flip = lambda f: lambda a,b: f(b,a)
13 curry = lambda f: lambda x: lambda y: f(x,y)
14 uncurry = lambda f: lambda p: f(*p)
15 def compose(*fs):
16     def g(x):
17         for f in reversed(fs):
18             x = f(x)
19         return x
20     return g
21 o = compose
22 def pipe(x,*fs):
23     for f in fs:
24         x = f(x)
25     return x
26 on = lambda f,g: lambda x,y: f(g(x),g(y))
27 # 2. pairs
28 fst = lambda p: p[0]
29 snd = lambda p: p[1]
30 swap = o(tuple,reversed)
31 # 3. sentinels
32 class _Sentinel:
33     __slots__ = ("_n",)
34     def __init__(self,n): self._n = n
35     def __repr__(self): return self._n
36     def __bool__(self): return False
37 NOTHING = _Sentinel("NOTHING")
38 FAIL = _Sentinel("FAIL")
39 _MISS = _Sentinel("_MISS")
40 # 4. arithmetic & logic
41 div = lambda a,b: a // b
42 mod = lambda a,b: a % b
43 halve = lambda n: n // 2
44 even = lambda n: n % 2 == 0
45 odd = lambda n: n % 2 == 1
46 otherwise = True
47 quot = lambda a,b: -(a // b) if (a < 0) != (b < 0) else a // b
48 rem = lambda a,b: a - b * quot(a,b)
49 quotRem = lambda a,b: (quot(a,b), rem(a,b))
50 signum = lambda x: (x > 0) - (x < 0)
51 succ = lambda x: chr(ord(x) + 1) if isinstance(x,str) else x + 1
```

```

52 pred = lambda x: chr(ord(x) - 1) if isinstance(x,str) else x - 1
53 # 5. folds, scans, unfolds
54 foldl = lambda f,xs,base=_MISS: _reduce(f,xs) if base is _MISS else _reduce(f,xs,base)
55 foldr = lambda f,xs,base: _reduce(flip(f),reversed(list(xs)),base)
56 foldl1 = foldl
57 foldr1 = lambda f,xs: _reduce(flip(f),reversed(list(xs)))
58 scanl = lambda f,xs,base=_MISS: list(_acc(xs,f) if base is _MISS else _acc(xs,f,initial=base))
59 scanl1 = scanl
60 scanr = lambda f,xs,base: scanl(flip(f),reversed(list(xs)),base)[::-1]
61 scanr1 = lambda f,xs: scanl(flip(f),reversed(list(xs)))[::-1]
62 def unfoldr(f,seed):
63     out = []
64     while (r := f(seed)) is not NOTHING:
65         out.append(r[0]); seed = r[1]
66     return out
67 def until(cond,f,x):
68     while not cond(x):
69         x = f(x)
70     return x
71 replicateM = lambda n,xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs],range(n),[[]])
72 subsequences = lambda xs: foldl(lambda acc,x: acc + [s + [x] for s in acc],list(xs),[[]])
73 # 6. streams
74 def iterate(f,x):
75     while True:
76         yield x
77         x = f(x)
78 take = lambda n,xs: list(_islice(iter(xs),n))
79 # 7. list basics
80 head = lambda xs: xs[0]
81 tail = lambda xs: xs[1:]
82 init = lambda xs: xs[:-1]
83 last = lambda xs: xs[-1]
84 drop = lambda n,xs: list(xs)[max(n,0):]
85 splitAt = lambda n,xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)
86 replicate = lambda n,x: [x] * n
87 elem = lambda x,xs: x in xs
88 notElem = lambda x,xs: x not in xs
89 null = lambda xs: len(xs) == 0
90 nub = o(list,dict.fromkeys)
91 enum = lambda xs,start=0: list(enumerate(xs,start))
92 pairwise = lambda xs: list(_pairwise(xs))
93 zip_ = lambda a,b: list(zip(a,b))
94 zip3 = lambda a,b,c: list(zip(a,b,c))
95 unzip = lambda ps: tuple(map(list,zip(*ps))) if ps else ([], [])
96 isPrefixOf = lambda p,xs: list(xs[:len(p)]) == list(p)
97 isSuffixOf = lambda p,xs: list(xs[len(xs) - len(p):]) == list(p)
98 lookup = lambda k,pairs: next((v for kk, v in pairs if kk == k),NOTHING)
99 stripPrefix = lambda p,xs: xs[len(p):] if isPrefixOf(p,xs) else NOTHING
100 # 8. higher-order lists
101 map_ = lambda f,xs: [f(x) for x in xs]
102 filter_ = lambda cond,xs: [x for x in xs if cond(x)]
103 concat = lambda xss: [x for xs in xss for x in xs]
104 concatMap = lambda f,xs: [y for x in xs for y in f(x)]
105 cross = lambda a,b: [(x, y) for x in a for y in b]
106 starmap = lambda f,pairs: [f(*p) for p in pairs]
107 zipWith = lambda f,a,b: [f(x,y) for x, y in zip(a,b)]
108 zipWith3 = lambda f,a,b,c: [f(x,y,z) for x, y, z in zip(a,b,c)]
109 find = lambda cond,xs: next((x for x in xs if cond(x)),NOTHING)
110 def partition(cond,xs):
111     yes, no = [], []
112     for x in xs:
113         (yes if cond(x) else no).append(x)
114     return (yes, no)
115 # 9. slicing & spans
116 def span(cond,xs):
117     xs = list(xs)
118     i = next((i for i, x in enumerate(xs) if not cond(x)),len(xs))
119     return (xs[:i], xs[i:])
120 break_ = lambda cond,xs: span(o(not_,cond),xs)
121 def takewhile(cond,xs):
122     for x in xs:
123         if not cond(x): return
124     yield x
125 def dropwhile(cond,xs):

```

```

126     it = iter(xs)
127     for x in it:
128         if not cond(x):
129             yield x
130             break
131     yield from it
132 def takeuntil(cond,xs):
133     for x in xs:
134         yield x
135         if cond(x): return
136 takeWhile = lambda cond,xs: list(takewhile(cond,xs))
137 dropWhile = lambda cond,xs: list(dropwhile(cond,xs))
138 inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)]
139 tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)]
140 def chunksOf(n,xs):
141     xs = list(xs)
142     return [xs[i:i + n] for i in range(0,len(xs),n)]
143 def windows(n,xs):
144     xs = list(xs)
145     return [xs[i:i + n] for i in range(len(xs) - n + 1)]
146 def groupBy(eq,xs):
147     out = []
148     for x in xs:
149         if out and eq(out[-1][0],x): out[-1].append(x)
150         else: out.append([x])
151     return out
152 group = partial(groupBy,lambda a,b: a == b)
153 def transpose(rows):
154     rows = [list(r) for r in rows]
155     out, i = [], 0
156     while any(i < len(r) for r in rows):
157         out.append([r[i] for r in rows if i < len(r)])
158         i += 1
159     return out
160 # 10. sorting & searching
161 sortOn = lambda f,xs: sorted(xs,key=f)
162 sortBy = lambda cmp,xs: sorted(xs,key=cmp_to_key(cmp))
163 maxOn = lambda f,xs: max(xs,key=f)
164 minOn = lambda f,xs: min(xs,key=f)
165 merge = lambda a,b: list(_hmerge(a,b))
166 # 11. strings
167 lines = str.splitlines
168 unlines = lambda ls: "".join(l + "\n" for l in ls)
169 words = str.split
170 unwords = " ".join
171 # 12. containers
172 def fromListWith(f,pairs):
173     d = {}
174     for k, v in pairs:
175         d[k] = f(v,d[k]) if k in d else v
176     return d
177 insertWith = lambda f,k,v,d: {**d, k: f(v,d[k]) if k in d else v}
178 unionWith = lambda f,a,b: {k: f(a[k],b[k]) if k in a and k in b else a[k] if k in a else b[k]
179                             for k in a.keys() | b.keys()}
180 def getpath(t,ks,default=NOTHING):
181     for k in ks:
182         if not isinstance(t,dict) or k not in t: return default
183         t = t[k]
184     return t
185 bag_diff = lambda a,b: {k: a[k] - b.get(k,0) for k in a if a[k] > b.get(k,0)}
186 bag_inter = lambda a,b: {k: v for k in a.keys() & b.keys() if (v := min(a[k],b[k]))}
187 bag_union = lambda a,b: {k: max(a.get(k,0),b.get(k,0)) for k in a.keys() | b.keys()}
188 bag_sub = lambda a,b: all(b.get(k,0) >= n for k,n in a.items())
189 # 13. Maybe
190 bind = lambda x,f: NOTHING if x is NOTHING else f(x)
191 isJust = lambda x: x is not NOTHING
192 isNothing = lambda x: x is NOTHING
193 fromMaybe = lambda default,x: default if x is NOTHING else x
194 listToMaybe = lambda xs: xs[0] if xs else NOTHING
195 maybeToList = lambda x: [] if x is NOTHING else [x]
196 catMaybes = lambda xs: [x for x in xs if x is not NOTHING]
197 mapMaybe = lambda f,xs: [y for y in map(f,xs) if y is not NOTHING]
198 def sequenceM(xs):
199     out = []

```

```

200     for x in xs:
201         if x is NOTHING: return NOTHING
202         out.append(x)
203     return out
204 traverseM = lambda f,xs: sequenceM(map_(f,xs))
205 maybe_get = lambda d,k: d.get(k,NOTHING)
206 # 14. Either
207 Ok = lambda v: ("ok", v)
208 Err = lambda e: ("err", e)
209 bindE = lambda x,f: x if x[0] == 'err' else f(x[1])
210 def sequenceE(xs):
211     out = []
212     for x in xs:
213         if x[0] == "err": return x
214         out.append(x[1])
215     return Ok(out)
216 traverseE = lambda f,xs: sequenceE(map_(f,xs))
217 def partitionEithers(xs):
218     oks, errs = [], []
219     for tag, v in xs:
220         (oks if tag == "ok" else errs).append(v)
221     return (oks, errs)
222 note = lambda msg,x: Err(msg) if x is NOTHING else Ok(x)
223 # 15. do-notation
224 def do(binder,is_fail):
225     def deco(gen_fn):
226         def run(*a,**kw):
227             g = gen_fn(*a,**kw)
228             def step(val):
229                 try:
230                     m = g.send(val)
231                     except StopIteration as e:
232                         return e.value
233                     return binder(m,step)
234             return step(None)
235         return run
236     return deco
237 doM = do(bind,lamba x: x is NOTHING)
238 doE = do(bindE,lamba x: x[0] == "err")
239 # 16. parser combinators
240 def doP(gen_fn):
241     def make(*a,**kw):
242         def parser(s):
243             g, val = gen_fn(*a,**kw), None
244             try:
245                 while True:
246                     r = g.send(val)(s)
247                     if r is FAIL: return FAIL
248                     val, s = r
249             except StopIteration as e:
250                 return (e.value, s)
251         return parser
252     return make
253 alt = lambda *ps: lambda s: next((r for r in (p(s) for p in ps) if r is not FAIL),FAIL)
254 def many(p):
255     def parser(s):
256         out = []
257         while (r := p(s)) is not FAIL and r[1] != s:
258             out.append(r[0]); s = r[1]
259         return (out, s)
260     return parser
261 def many1(p):
262     def parser(s):
263         r = p(s)
264         if r is FAIL: return FAIL
265         rest = many(p)(r[1])
266         return ([r[0]] + rest[0], rest[1])
267     return parser
268 def sepBy(p,sep):
269     def pair(s):
270         r = sep(s)
271         return FAIL if r is FAIL else p(r[1])
272     def parser(s):
273         r = p(s)

```

```

274         if r is FAIL: return ([], s)
275         rest = many(pair)(r[1])
276         return ([r[0]] + rest[0], rest[1])
277     return parser
278 def lit(c):
279     def parser(s):
280         s = s.lstrip()
281         return (c, s[1:]) if s[:1] == c else FAIL
282     return parser
283 def rx(pat,conv=identity):
284     r = _re.compile(pat)
285     def parser(s):
286         s = s.lstrip()
287         m = r.match(s)
288         return (conv(m.group()), s[m.end():]) if m else FAIL
289     return parser
290 def chainl1(p,ops):
291     opp = alt(*[lit(c) for c in ops])
292     def parser(s):
293         vr = p(s)
294         while vr is not FAIL:
295             op = opp(vr[1])
296             if op is FAIL: return vr
297             w = p(op[1])
298             if w is FAIL: return vr
299             vr = (ops[op[0]](vr[0],w[0]), w[1])
300         return FAIL
301     return parser
302 def runParser(p,s):
303     r = p(s)
304     if r is FAIL: return Err(f"no parse: {s!r}")
305     if r[1].strip(): return Err(f"unconsumed input: {r[1].strip()!r}")
306     return Ok(r[0])
307 # 17. monoids
308 Monoid = lambda empty,op: (empty, op)
309 mconcat = lambda m,xs: foldl(m[1],xs,m[0])
310 foldMap = lambda f,m,xs: mconcat(m,[f(x) for x in xs])
311 both = lambda m1,m2: Monoid((m1[0], m2[0]),
312     lambda a,b: (m1[1](a[0],b[0]), m2[1](a[1],b[1])))
313 Sum = Monoid(0,add)
314 Product = Monoid(1,mul)
315 All = Monoid(True,lambda a,b: a and b)
316 Any = Monoid(False,lambda a,b: a or b)
317 MaxM = Monoid(float("-inf"),max)
318 MinM = Monoid(float("inf"),min)
319 ListM = Monoid([],add)
320 First = Monoid(NOTHING,lambda a,b: b if a is NOTHING else a)
321 Last = Monoid(NOTHING,lambda a,b: a if b is NOTHING else b)

```


Chapter 2

Point-Free Foundations

A function that names its argument commits to one shape of input; a function built from other functions (`compose`, `on`, `partial`) stays a transformation, reusable wherever the types line up. This chapter’s five one-liners (`identity`, `const`, `flip`, `curry`, `uncurry`) are the adapters that make composition possible at all – they change nothing about the values, only the SHAPE of the call. Two sentinels close the chapter: `NOTHING` means “no answer”, and `FAIL` means “the parser gave up” – both real objects, never `None`, so a function can return an ordinary `None` as genuine data without it being mistaken for absence.

2.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest` referee.

1.1 Mirror a pair

drill

Write `mirror(p)`: the pair with its elements exchanged, point-free. Applying it twice must give the original back.

```
mirror(p) -> pair
|>>> mirror((1, 2))
| (2, 1)
|>>> mirror(mirror((3, 4)))
| (3, 4)
```

Solution.

```
| mirror = o(tuple, reversed)
```

Note. Reads as a sentence: tuple of reversed. No argument ever named.

1.2 Record projections

drill

A leaderboard record is a (name, score) pair. Write `name_of` and `score_of` point-free, without lambdas.

```
name_of(r) -> value; score_of(r) -> value
|>>> name_of(('ada', 95))
| 'ada'
|>>> score_of(('ada', 95))
| 95
```

Solution.

```
| name_of, score_of = itemgetter(0), itemgetter(1)
```

Note. `operator.itemgetter` is Python’s `fst/snd` – and it slots straight into `map` and `sorted(key=...)`.

1.3 Default when missing

drill

Write `port_of(cfg)`: the 'port' entry of a config dict, defaulting to 8080 only when the key is absent. A key holding `None` is real data and must survive.

`port_of(cfg) -> value`

```
>>> port_of({'port': 9000})
9000
>>> port_of({})
8080
>>> port_of({'port': None}) is None
True
```

Solution.

```
| port_of = lambda cfg: fromMaybe(8080, maybe_get(cfg, 'port'))
```

Note. The prelude course's `or_default` cannot pass the third test: with `None-as-Nothing`, `None` IS the miss. The `NOTHING` sentinel is the fix.

1.4 Clamp to a range

applied

Write `clamp(lo, hi)`: a FUNCTION that forces a number into `[lo, hi]`. Build it from `max` and `min` with partial application – no `lambda` in the returned function.

`clamp(lo, hi) -> (x -> value)`

```
>>> clamp(0, 10)(15)
10
>>> clamp(0, 10)(-5)
0
>>> clamp(0, 10)(7)
7
```

Solution.

```
| clamp = lambda lo, hi: o(partial(max, lo), partial(min, hi))
```

Note. Right to left: cap above with `min(hi, .)`, then floor with `max(lo, .)`. Sections via `partial`.

1.5 Twice

drill

Write `twice(f)`: the function that applies `f` two times.

`twice(f) -> (x -> value)`

```
>>> twice(lambda n: n * 3)(2)
18
>>> twice(str.strip)(' hi ')
'hi'
```

Solution.

```
| twice = lambda f: o(f, f)
```

Note. `twice = o` on the diagonal. `twice(twice(f))` is `f` four times – combinators compose with themselves.

Chapter 3

Folds, Scans and Unfolds

A fold collapses a list to one value; a scan is a fold that keeps its history; an unfold grows a list from a seed instead of consuming one. `scanl` here is *exactly* `itertools.accumulate` – naming it the Haskell way does not change what runs. `unfoldr` is the fold’s mirror image: given a step function that returns `(value, next_seed)` or `NOTHING` to stop, it produces the whole list, which is why balances, digit sequences and Collatz chains all turn out to be one `unfoldr` call apiece. `iterate` is `unfoldr`’s lazy, infinite cousin: it never stops on its own, so `take` is what makes it usable.

3.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

2.1 Parity census

drill

Count evens and odds as a pair (evens, odds), in ONE pass, point-free at the top level.

`parity_counts(xs) -> (evens, odds)`

```
>>> parity_counts([1, 2, 3, 4])
(2, 2)
>>> parity_counts([])
(0, 0)
>>> parity_counts([-2, -3])
(1, 1)
```

Solution.

```
| parity_counts = partial(foldMap, lambda x: (1 - x % 2, x % 2), both(Sum, Sum))
```

Note. `both(Sum, Sum)` is two counters riding one fold; `partial` freezes the fold into a name. The `[]` case is free: monoid identity.

2.2 Running balance

drill

An account starts at 0; deposits are positive, withdrawals negative. Return every intermediate balance including the start.

`balances(txs) -> [balance]`

```
>>> balances([100, -30, -30, 5])
[0, 100, 70, 40, 45]
>>> balances([])
[0]
```

Solution.

```
| balances = lambda txs: scanl(add, txs, 0)
```

Note. `Scan` = fold that keeps its diary. `operator.add` instead of a `lambda`.

2.3 Countdown

drill

Write `countdown(n)`: the list `n, n-1, ..., 1` grown by `unfoldr`, point-free at the top level.

`countdown(n) -> [int]`

```
>>> countdown(4)
[4, 3, 2, 1]
>>> countdown(0)
[]
```

Solution.

```
countdown = partial(unfoldr, lambda n: NOTHING if n == 0 else (n, n - 1))
```

Note. `unfoldr` is the anamorphism: seed in, list out, `NOTHING` stops it.

2.4 Digits of a number

applied

Write `digits(n)`: the base-10 digits of a non-negative integer, most significant first. Zero has one digit.

`digits(n) -> [int]`

```
>>> digits(407)
[4, 0, 7]
>>> digits(0)
[0]
```

Solution.

```
digits = lambda n: unfoldr(lambda k: NOTHING if k == 0 else (k % 10, k // 10), n)[::-1] or [0]
```

Note. `Unfold` peels least-significant first; `reverse` restores reading order. `'or [0]'` patches the empty `unfold` at zero.

2.5 Collatz flight length

challenge

Write `collatz_len(n)`: how many numbers the Collatz sequence visits from `n` down to 1, inclusive. Even halves; odd triples-plus-one.

`collatz_len(n) -> int`

```
>>> collatz_len(6)
9
>>> collatz_len(1)
1
```

Solution.

```
collatz_len = lambda n: 1 + len(unfoldr(lambda k: NOTHING if k == 1 else (k, k // 2 if k % 2 == 0 else 3 * k + 1), n))
```

Note. The `unfold` emits everything before the fixed point; `+1` counts the 1 itself. No mutation, no `while`.

Chapter 4

The List Gaps

The largest chapter because it is the least glamorous: the ordinary list and dict work that every program needs, named consistently instead of reached for ad hoc. `windows` and `groupBy` both slide a view across a sequence, but for different reasons – one for a fixed-width lookback (moving averages), one for runs of equal elements (RLE, compression). `transpose` is ragged-safe on purpose: short rows simply drop out of later columns instead of raising, so uneven data never needs a special case. `fromListWith` and `unionWith` are the two dict-building folds worth memorising – almost every “group these” or “merge these two counters” problem is one of them with the right combining function.

4.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

3.1 Run-length encoding

drill

Compress a sequence into (element, count) pairs per maximal run.

```
rle(xs) -> [(x, n)]

>>> rle('aaabbc')
[('a', 3), ('b', 2), ('c', 1)]
>>> rle([])
[]
```

Solution.

```
rle = lambda xs: [(g[0], len(g)) for g in groupBy(eq, xs)]
```

Note. `groupBy(eq, .)` does the segmentation; the comprehension just labels each run. `operator.eq`, no `lambda`.

3.2 Moving average

applied

Write `mavg(n, xs)`: the `n`-point moving average of a signal.

```
mavg(n, xs) -> [float]

>>> mavg(3, [1, 2, 3, 4, 5])
[2.0, 3.0, 4.0]
>>> mavg(3, [1, 2])
[]
```

Solution.

```
mavg = lambda n, xs: [sum(w) / n for w in windows(n, xs)]
```

Note. `windows` names the sliding view; the arithmetic stays arithmetic. Too-short input yields no windows – correct for free.

3.3 Pagination

drill

Write `paginate(n, xs)`: pages of at most `n` items, last page short.

```
paginate(n, xs) -> [[x]]
```

```
>>> paginate(2, [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
```

Solution.

```
| paginate = chunksOf
```

Note. Recognize-it drill: pagination IS `chunksOf`.

3.4 Column sums, ragged

applied

Sensor rows arrive with trailing dropouts. Sum each column over the rows that reached it – point-free.

```
colsums(rows) -> [sum]
```

```
>>> colsums([[1, 2, 3], [4, 5]])
[5, 7, 3]
>>> colsums([])
[]
```

Solution.

```
| colsums = o(list, partial(map, sum), transpose)
```

Note. Ragged-safe transpose turns dropouts into short columns instead of exceptions; then it is just `map sum`.

3.5 Indentation split

drill

Split a line into (leading whitespace, rest) as strings, one pass.

```
indent_split(s) -> (str, str)
```

```
>>> indent_split('  x = 1')
('  ', 'x = 1')
>>> indent_split('top')
('', 'top')
```

Solution.

```
| indent_split = lambda s: tuple(''.join(p) for p in span(str.isspace, s))
```

Note. `span` is the two-sided `takewhile`; the `join` restores strings from character lists.

Chapter 5

Maybe, Done Right

`None` is Python’s “no value”, which is exactly the problem: a function cannot then return `None` to mean a REAL answer without creating an ambiguity. `NOTHING` fixes this by being a distinct object that is never a legitimate value – `maybe_get` returns `None` for a key whose stored value truly is `None`, and `NOTHING` only when the key is missing outright. `bind` chains failable steps by short-circuiting the moment one returns `NOTHING`; `sequenceM` and `traverseM` extend that to a whole list, all-or-nothing. Once a pipeline’s failure mode is a value instead of an exception, the pipeline composes like any other function.

5.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

4.1 Safe head

drill

Write `headM(xs)`: the first element, or `NOTHING` for an empty sequence. A leading `None` is data.

`headM(xs) -> Maybe value`

```
>>> headM([7, 8])
7
>>> headM([]) is NOTHING
True
>>> headM([None]) is None
True
```

Solution.

```
headM = lambda xs: xs[0] if xs else NOTHING
```

Note. The third test is the whole point of the sentinel: `None` rides through as a value.

4.2 Deep dictionary path

applied

Write `pathM(d, ks)`: follow a key path through nested dicts; `NOTHING` if any hop is missing. A stored `None` must survive the trip.

`pathM(d, ks) -> Maybe value`

```
>>> d = {'db': {'host': 'local', 'pool': None}}
>>> pathM(d, ['db', 'host'])
'local'
>>> pathM(d, ['db', 'pool']) is None
True
>>> pathM(d, ['db', 'user']) is NOTHING
True
```

Solution.

```
pathM = lambda d, ks: foldl(lambda m, k: bind(m, lambda t: maybe_get(t, k)), ks, d)
```

Note. A path lookup is a FOLD of binds over the keys – the `Maybe` monad’s signature move.

4.3 Salvage the parseable

drill

Given strings, keep the ones that are integers, as ints, dropping the rest.

```
salvage_ints(ss) -> [int]
```

```
>>> salvage_ints(['3', 'x', '-7', ''])
[3, -7]
```

Solution.

```
salvage_ints = lambda ss: catMaybes([int(s) if s.lstrip('-').isdigit() and s else NOTHING for s in ss])
```

Note. Map into Maybe, then catMaybes sweeps the failures out – mapMaybe in two named steps.

4.4 All keys or nothing

applied

Write need(d, ks): the values for ks in order, or NOTHING if ANY key is absent.

```
need(d, ks) -> Maybe [value]
```

```
>>> need({'a': 1, 'b': 2}, ['a', 'b'])
[1, 2]
>>> need({'a': 1}, ['a', 'b']) is NOTHING
True
```

Solution.

```
need = lambda d, ks: sequenceM([maybe_get(d, k) for k in ks])
```

Note. sequenceM is the all-or-nothing gate: one miss poisons the batch.

4.5 First responder

challenge

Several config sources; write first_set(dict, k): the value from the FIRST dict that defines k at all – even as None – else NOTHING.

```
first_set(dict, k) -> Maybe value
```

```
>>> first_set([{}], {'x': None}, {'x': 3}), 'x') is None
True
>>> first_set([{}], {}, 'x') is NOTHING
True
```

Solution.

```
first_set = lambda dict, k: mconcat(First, [maybe_get(d, k) for d in dict])
```

Note. First’s identity is NOTHING, so ‘defined as None’ beats ‘undefined’ – exactly the override semantics config layering needs.

Chapter 6

Either: Failure With a Why

Maybe answers “did it work?”; Either answers that AND “why not?”. `Ok/Err` are tagged pairs – (“ok”, `v`) or (“err”, `why`) – so `bindE` can short-circuit exactly like `bind` while still carrying the failure’s reason to wherever the pipeline is finally inspected. `sequenceE` and `traverseE` are Either’s all-or-nothing gates, and `note` is the bridge from the last chapter: it turns a `NOTHING` into an `Err` with a message attached, for the moment a Maybe pipeline needs to explain itself to a caller.

6.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

5.1 Lift the miss into a reason

drill

Write `get_e(d, k)`: `Ok(value)` or `Err` naming the missing key.

`get_e(d, k) -> Either`

```
>>> get_e({'a': 1}, 'a')
('ok', 1)
>>> get_e({}, 'user')
('err', "missing key: 'user'")
```

Solution.

```
get_e = lambda d, k: note(f"missing key: {k!r}", maybe_get(d, k))
```

Note. `note` is the Maybe-to-Either bridge: same short-circuit, now with a why.

5.2 Parse then bound

applied

Write `to_port(s)`: parse a decimal string and demand 1..65535, each failure carrying its own message.

`to_port(s) -> Either`

```
>>> to_port('8080')
('ok', 8080)
>>> to_port('http')
('err', "not a number: 'http'")
>>> to_port('99999')
('err', 'out of range: 99999')
```

Solution.

```
to_port = lambda s: bindE(
    note(f"not a number: {s!r}", int(s) if s.isdigit() else NOTHING),
    lambda n: Ok(n) if 1 <= n <= 65535 else Err(f"out of range: {n}"))
```

Note. Two failure modes, two messages, one `bindE`. The caller pattern-matches once.

5.3 First error speaks

drill

Validate a batch: all Ok values, or the FIRST Err verbatim.

`all_ports(ss) -> Either`

```
>>> all_ports(['80', '443'])
('ok', [80, 443])
>>> all_ports(['80', 'x', 'y'])
('err', "not a number: 'x'")
```

Solution.

```
| all_ports = lambda ss: sequenceE([to_port(s) for s in ss])
```

Note. `sequenceE` = `sequenceM` that can explain itself.

5.4 Division with a why

drill

Write `div_e(a, b)`: the quotient, or an error naming the offense.

`div_e(a, b) -> Either`

```
>>> div_e(10, 4)
('ok', 2.5)
>>> div_e(1, 0)
('err', 'division by zero')
```

Solution.

```
| div_e = lambda a, b: Err('division by zero') if b == 0 else Ok(a / b)
```

Note. Exceptions escape; Either values travel. Choose Either at module boundaries.

Chapter 7

do-Notation

Chaining `bind` calls by hand nests one call inside the next, one level per step – the “bind pyramid”. `do` rebuilds that same chain out of an ordinary generator function: each `yield` is one `bind`, and the value sent back in is what the step would have returned. `doM` wires this to the Maybe monad, `doE` to Either – same decorator, same generator shape, different short-circuit rule. The win is not cleverness for its own sake: a five-step Maybe pipeline written with `doM` reads top to bottom, in the order the steps actually happen, with the failure path made invisible instead of handled at every line.

7.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest` referee.

6.1 Config reader, flat

drill

Using `doM`, read host and port from a dict; NOTHING if either is absent.

`hostport(d) -> Maybe (host, port)`

```
>>> hostport({'host': 'h', 'port': 80})
('h', 80)
>>> hostport({'host': 'h'}) is NOTHING
True
```

Solution.

```
@doM
def hostport(d):
    h = yield maybe_get(d, 'host')
    p = yield maybe_get(d, 'port')
    return (h, p)
```

Note. Compare the bind pyramid this replaces: each `yield` is one `»=`, and the failure path is invisible.

6.2 Join across two tables

applied

`users` maps id to name; `emails` maps name to address. Using `doM`, resolve an id to its address.

`address(users, emails, uid) -> Maybe str`

```
>>> u, e = {1: 'ada'}, {'ada': 'ada@x.io'}
>>> address(u, e, 1)
'ada@x.io'
>>> address(u, e, 2) is NOTHING
True
```

Solution.

```
@doM
def address(users, emails, uid):
```

```

    name = yield maybe_get(users, uid)
    return (yield maybe_get(emails, name))

```

Note. A two-hop join is two yields. The second bind consumes the first's value by name, not by nesting.

6.3 Validated record

applied

Using doE, build a user record from a form dict: name must be non-empty, age must parse and land in 0..130. Each rejection explains itself.

mk_user(form) -> Either dict

```

>>> mk_user({'name': 'ada', 'age': '36'})
('ok', {'name': 'ada', 'age': 36})
>>> mk_user({'name': '', 'age': '36'})
('err', 'empty name')
>>> mk_user({'name': 'ada', 'age': 'old'})
('err', "bad age: 'old'")

```

Solution.

```

@doE
def mk_user(form):
    name = yield (Ok(form.get('name', '')) if form.get('name') else Err('empty name'))
    raw = form.get('age', '')
    age = yield note(f"bad age: {raw!r}", int(raw) if raw.isdigit() else NOTHING)
    yield (Ok(age) if age <= 130 else Err(f'implausible age: {age}'))
    return Ok({'name': name, 'age': age})

```

Note. A bare yield with no binding is a guard: run the check, keep no value. The happy path reads top to bottom.

6.4 Chained arithmetic

drill

Using doE and div_e from 5.4, compute $a/b + c/d$ with the first bad divisor reported.

sum_of_ratios(a, b, c, d) -> Either

```

>>> sum_of_ratios(1, 2, 1, 4)
('ok', 0.75)
>>> sum_of_ratios(1, 0, 1, 4)
('err', 'division by zero')

```

Solution.

```

@doE
def sum_of_ratios(a, b, c, d):
    x = yield div_e(a, b)
    y = yield div_e(c, d)
    return Ok(x + y)

```

Note. Error plumbing gone; only the arithmetic remains.

Chapter 8

Parser Combinators

A parser is a function from a string to `(value, rest)` or `FAIL` – once that shape is fixed, parsers compose like any other function. `doP` threads the remaining input through a generator the same way `doM` threads a `Maybe`; `alt` tries alternatives in order; `many` and `sepBy` handle repetition and separator-delimited lists without recursion. `chainl1` deserves special attention: it folds a sequence of `p (op p) * LEFT`, which is exactly how left-associative arithmetic is supposed to parse, and it is the one combinator in this chapter doing real algorithmic work rather than just gluing others together.

8.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

7.1 Comma-separated integers

drill

Parse a comma-separated integer list, whitespace-blind, whole input consumed.

`int_list(s) -> Either [int]`

```
>>> int_list('1, -2 , 3')
('ok', [1, -2, 3])
>>> int_list('1, x')
('err', "unconsumed input: ', x'")
```

Solution.

```
int_list = lambda s: runParser(sepBy(rx(r' -?\d+', int), lit(',')), s)
```

Note. `sepBy` handles the fencepost (separators BETWEEN items); `runParser` demands the input be spent and lands in `Either`.

7.2 key=value line

drill

Parse one 'key = value' assignment into a pair; keys are words, values run to end of line.

`kv(s) -> Either (k, v)`

```
>>> kv('host = 10.0.0.1')
('ok', ('host', '10.0.0.1'))
>>> kv('= oops')[0]
'err'
```

Solution.

```
@doP
def _kv():
    k = yield rx(r'\w+')
    yield lit('=')
    v = yield rx(r'[\n]+'), str.strip)
    return (k, v)
kvline = _kv()
kv = lambda s: runParser(kvline, s)
```

Note. Three yields, no rest-threading anywhere. kvline is reusable – chapter 9’s INI reader is built on it.

7.3 CSV row

drill

Split one unquoted CSV row into stripped fields.

```
csv_row(s) -> Either [str]

>>> csv_row('ada, 95, pass')
('ok', ['ada', '95', 'pass'])
```

Solution.

```
csv_row = lambda s: runParser(sepBy(rx(r'[^,\n]+'), str.strip), lit(','), s)
```

Note. The same sepBy skeleton as 7.1 with a different token – combinator reuse is the whole pitch.

7.4 ISO date

applied

Parse YYYY-MM-DD into an (y, m, d) integer triple, rejecting month 13.

```
iso_date(s) -> Either (y, m, d)

>>> iso_date('2026-09-17')
('ok', (2026, 9, 17))
>>> iso_date('2026-13-01')
('err', 'bad month: 13')
```

Solution.

```
num = lambda n: rx(r'\d{%d}' % n, int)
@doP
def _date():
    y = yield num(4)
    yield lit('-')
    m = yield num(2)
    yield lit('-')
    d = yield num(2)
    return (y, m, d)
iso_date = lambda s: bindE(runParser(_date(), s),
    lambda t: Ok(t) if 1 <= t[1] <= 12 else Err(f'bad month: {t[1]}'))
```

Note. Parse the shape with doP, judge the meaning with bindE – syntax and semantics in separate layers.

7.5 Sums and differences

challenge

Parse and evaluate a left-associative +/- chain over integers.

```
sums(s) -> Either int

>>> sums('10 - 3 - 2 + 1')
('ok', 6)
>>> sums('10 -')[0]
'err'
```

Solution.

```
sums = lambda s: runParser(chainl1(rx(r'[-?\d+]', int), {'+': add, '-': sub}), s)
```

Note. chainl1 IS left-associativity-as-a-fold; operator.add/sub keep it point-free. Chapter 9 grows this into the full calculator.

Chapter 9

Monoids at Work

A monoid is nothing but an identity element paired with an associative combiner, reified as the pair `(empty, op)` – naming it as `DATA` means one engine, `mconcat`, folds every instance, and `foldMap` fuses “measure each element” with “combine the measurements” into one pass. `both` is the chapter’s real payoff: it pairs two monoids into one, so a max and a min, or a count and a total, fall out of a `SINGLE` traversal instead of two separate loops. That matters most exactly when it looks like it should not – a one-shot iterator or a huge file that can only be read once.

9.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

8.1 Min and max, one pass

drill

Both extremes of a numeric stream in a single traversal, point-free.

```
extremes(xs) -> (min, max)
```

```
>>> extremes([3, 1, 4, 1, 5])
(1, 5)
```

Solution.

```
extremes = partial(foldMap, lambda x: (x, x), both(MinM, MaxM))
```

Note. `both` fuses two folds into one pass – what matters when `xs` is a one-shot iterator or a large file.

8.2 Stream statistics

applied

Count, total and maximum of a stream in ONE pass: `((count, total), maximum)`.

```
stats(xs) -> ((n, total), mx)
```

```
>>> stats([3, 1, 4])
((3, 8), 4)
```

Solution.

```
stats = partial(foldMap, lambda x: ((1, x), x), both(both(Sum, Sum), MaxM))
```

Note. `both` nests: a monoid of monoids. The tuple shape of the answer mirrors the shape of the fold.

8.3 Config layering

drill

defaults, file, cli – highest layer that `DEFINES` a key wins, and defining it as `None` counts. Point-free over the layer list.

```
effective(layers, k) -> Maybe value
```

```
>>> layers = [{'v': 1}, {}, {'v': None}]
>>> effective(layers, 'v') is None
```

```
True
>>> effective(layers, 'x') is NOTHING
True
```

Solution.

```
effective = lambda layers, k: mconcat>Last, [maybe_get(d, k) for d in layers])
```

Note. Last with a NOTHING identity is override semantics verbatim. Same skeleton as 4.5 with the opposite monoid.

8.4 Flatten

drill

Concatenate a list of lists by monoid fold.

```
flatten(xss) -> [x]
```

```
>>> flatten([[1], [], [2, 3]])
[1, 2, 3]
```

Solution.

```
flatten = partial(mconcat, ListM)
```

Note. Recognize-it: concatenation is the list monoid's mconcat. (For huge inputs reach for itertools.chain – this one is quadratic.)

Chapter 10

Capstone Labs

No new vocabulary – this chapter is the payoff for the previous eight. Each problem reaches back for tools from more than one chapter at once: parsing with `doP`, then judging the parsed value with `bindE`; folding a monoid across a batch validated with `doE`. If a chapter's tools felt like isolated tricks on first reading, this is where they stop being isolated.

10.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

9.1 The algebraic calculator

lab

Full expression evaluator: `+` `-` `*` `/`, precedence, LEFT associativity, unary minus, parentheses, decimals, whitespace; Either out.

`calc(s) -> Either float`

```
>>> calc('2 * (3 + 4) - -5')
('ok', 19.0)
>>> calc('8 - 3 - 2')
('ok', 3.0)
>>> calc('2 * (3 +)')[0]
'err'
```

Solution.

```
number = rx(r'[-?\d+(\.\d+)??', float)
@doP
def _paren():
    yield lit('(')
    v = yield expr
    yield lit(')')
    return v
unary = lambda s: alt(number, _paren())(s)
term = chain1(unary, {'*': mul, '/': lambda a, b: a / b})
expr = chain1(term, {'+': add, '-': sub})
calc = partial(runParser, expr)
```

Note. Eight grammar lines. Every layer earns its keep: `doP` threads state, `chain1` folds associativity, `runParser` explains failure. 5000-term chains run in constant stack.

9.2 INI reader

lab

Parse `[section]` headers with `key = value` bodies into a dict of dicts.

`ini(s) -> Either section: k: v`

```
>>> t = '[db]\nhost = local\nport = 5432\n[app]\ndebug = true'
>>> ini(t)
('ok', {'db': {'host': 'local', 'port': '5432'}, 'app': {'debug': 'true'}})
```

Solution.

```

@doP
def _kvi():
    k = yield rx(r'\w+')
    yield lit('=')
    v = yield rx(r'^\n\[ ]+', str.strip)
    return (k, v)
@doP
def _section():
    yield lit '['
    name = yield rx(r'\w+')
    yield lit(' ')
    kvs = yield many(_kvi())
    return (name, dict(kvs))
ini = lambda s: bindE(runParser(many(_section()), s), o(Ok, dict))

```

Note. The 7.2 skeleton, nested once. many loops, so thousand-key files cost no stack; bindE lifts the final dict without unwrapping by hand.

9.3 Log triage*lab*

Lines are 'LEVEL message'. In ONE pass report ((errors, total), first ERROR message or NOTHING).

trriage(lines) -> ((errors, total), first_err)

```

>>> triage(['INFO up', 'ERROR db down', 'ERROR retry', 'INFO ok'])
((2, 4), 'db down')
>>> triage(['INFO up'])[1] is NOTHING
True

```

Solution.

```

tag = lambda ln: (lambda lvl, _, msg: ((lvl == 'ERROR', 1),
                                     msg if lvl == 'ERROR' else NOTHING))(*ln.partition(' '))
trriage = partial(foldMap, tag, both(both(Sum, Sum), First))

```

Note. Counting and first-witness are DIFFERENT monoids riding the same pass. Adding a new statistic is one more both, not another loop.

9.4 Batch admission*lab*

Admit a batch of form dicts with 6.3's mk_user: all records, or the first rejection verbatim.

admit(forms) -> Either [dict]

```

>>> admit([{'name': 'ada', 'age': '36'}, {'name': 'bob', 'age': '41'}])
('ok', [{'name': 'ada', 'age': 36}, {'name': 'bob', 'age': 41}])
>>> admit([{'name': 'ada', 'age': '36'}, {'name': '', 'age': '2'}])
('err', 'empty name')

```

Solution.

```

admit = o(sequenceE, partial(map, mk_user), list)

```

Note. Validation composes: per-record doE, batch-level sequenceE, glued point-free. The whole intake pipeline is one line.

Part II

Compare and Contrast: Imperative vs Functional

About this part

Ten small apps, each written twice over the same inputs: once imperative, once with `haskell.py`, with a `__main__` block that ASSERTS the two agree. Every listing below is verified to actually run and agree before this book is built – what you read is what was executed.

	imperative	functional
state	mutable slots you maintain	threaded by the combinator
errors	exceptions / early return	values (NOTHING, Err) that compose
no value yet	hand-rolled sentinel per site	NOTHING, once, in the library
passes over data	one loop per concern	monoids fuse concerns into one pass
extension	edit inside the loop	add a stage / another both

Chapter 11

calculator

evaluate '2 * (3 + 4) - 5' both ways

Imperative

```
def calc_imp(s):
    toks, i = [], 0
    while i < len(s):
        c = s[i]
        if c.isspace(): i += 1; continue
        if c in '+-*/()': toks.append(c); i += 1; continue
        if c.isdigit():
            j = i
            while j < len(s) and (s[j].isdigit() or s[j] == '.'): j += 1
            toks.append(float(s[i:j])); i = j; continue
        return ('err', f'bad char {c!r}')

    class P:
        pos = 0
    def peek(): return toks[P.pos] if P.pos < len(toks) else None
    def unary():
        t = peek()
        if t == '-': P.pos += 1; return -unary()
        if t == '(':
            P.pos += 1; v = expr()
            if peek() != ')': raise ValueError('expected ')
            P.pos += 1; return v
        if isinstance(t, float): P.pos += 1; return t
        raise ValueError(f'unexpected {t!r}')
    def term():
        v = unary()
        while peek() in ('*', '/'):
            op = peek(); P.pos += 1; w = unary()
            v = v * w if op == '*' else v / w
        return v
    def expr():
        v = term()
        while peek() in ('+', '-'):
            op = peek(); P.pos += 1; w = term()
            v = v + w if op == '+' else v - w
        return v

    try:
        v = expr()
        return ('ok', v) if P.pos == len(toks) else ('err', 'unconsumed input')
    except ValueError as e:
        return ('err', str(e))
```

Functional

```

number = rx(r'-?\d+(\.\d+)?', float)
@doP
def _paren():
    yield lit('(')
    v = yield expr_fn
    yield lit(')')
    return v
unary_fn = lambda s: alt(number, _paren())(s)
term_fn = chainl1(unary_fn, {'*': mul, '/': lambda a, b: a / b})
expr_fn = chainl1(term_fn, {'+': add, '-': sub})
calc_fn = partial(runParser, expr_fn)

```

Discussion

The imperative version’s mutable cursor (`class P: pos = 0`) is a common workaround for parsing in Python: state lives on an object because a closure can’t reassign an outer `int` without `nonlocal` threaded through every helper. The functional version needs no cursor at all – `doP` threads the remaining input as an ordinary return value, and `chainl1` handles left-associativity as a fold instead of a hand-written `while` loop. Errors ride two different rails, too: exceptions in the imperative version, an `Either` in the functional one – and only the `Either` can be inspected, logged, or handed to another function without a `try/except` at every call site.

Chapter 12

gradebook

validate 'name,score' rows, then one-pass class stats

Imperative

```
def admit_imp(rows):
    out = []                                # accumulate + early return
    for row in rows:
        name, _, raw = row.partition(',')
        if not name:
            return ('err', f'empty name in {row!r}')
        if not raw.strip().isdigit():
            return ('err', f'bad score in {row!r}')
        score = int(raw)
        if score > 100:
            return ('err', f'score > 100 in {row!r}')
        out.append((name, score))
    n = tot = 0                             # second loop for stats
    best = float('-inf')
    for _, s in out:
        n += 1; tot += s
        if s > best: best = s
    return ('ok', (out, (n, tot), best))
```

Object-Oriented

```
class GradeBook:
    """The same job, encapsulated: a mutable object that grows by .add(row),
    raising on the first bad row instead of returning an Err tuple."""
    def __init__(self):
        self.rows = []

    def add(self, row):
        name, _, raw = row.partition(',')
        if not name:
            raise ValueError(f'empty name in {row!r}')
        if not raw.strip().isdigit():
            raise ValueError(f'bad score in {row!r}')
        score = int(raw)
        if score > 100:
            raise ValueError(f'score > 100 in {row!r}')
        self.rows.append((name, score))
        return self                        # chainable: book.add(r1).add(r2)

    def stats(self):
```

```

        n = len(self.rows)
        tot = sum(s for _, s in self.rows)
        best = max((s for _, s in self.rows), default=float('-inf'))
        return (n, tot), best

def admit_oop(rows):
    book = GradeBook()
    try:
        for row in rows:
            book.add(row)
    except ValueError as e:
        return ('err', str(e))
    return ('ok', (book.rows, *book.stats()))

```

Functional

```

@doE
def _row(row):
    name, _, raw = row.partition(',')
    yield Ok(name) if name else Err(f'empty name in {row!r}')
    s = yield note(f'bad score in {row!r}', int(raw) if raw.strip().isdigit() else NOTHING)
    yield Ok(s) if s <= 100 else Err(f'score > 100 in {row!r}')
    return Ok((name, s))

stats = partial(foldMap, lambda p: ((1, p[1]), p[1]), both(both(Sum, Sum), MaxM))
admit_fn = lambda rows: bindE(traverseE(_row, rows),
                              lambda ps: Ok((ps, *stats(ps))))

```

Discussion

Compare all three. The imperative version accumulates into a list and returns early on a bad row, then needs a SECOND loop afterward just to compute stats. The object-oriented version is the natural instinct for validation-heavy code – a class that grows by `.add()` and raises on trouble – but it still needs a `try/except` at the call site, and its `.stats()` re-walks the same rows the loop already built. The functional version fuses validation (`traverseE`) and the one-pass fold (`foldMap` with `both` nested twice) into two lines: one pass builds the valid rows AND the stats at once, because `both` fuses monoids instead of running loops back to back.

Chapter 13

layers

layered config: defaults < file < cli; None is a real value

Imperative

```
_UNSET = object()                                # invent a private sentinel,
def effective_imp(layers, k):                      # because None must survive
    val = _UNSET
    for d in layers:
        if k in d:
            val = d[k]
    return NOTHING if val is _UNSET else val

def merged_imp(layers):
    out = {}
    for d in layers:
        for k, v in d.items():
            out[k] = v
    return out
```

Functional

```
effective_fn = lambda layers, k: mconcat>Last, [maybe_get(d, k) for d in layers])
merged_fn    = lambda layers: foldl(lambda a, b: unionWith(lambda old, new: new, a, b),
                                   layers, {})
```

Discussion

This is the one example where “just use a config object” would recreate the exact bug the Maybe chapter exists to prevent: a plain class doing `self.proxy = self.proxy` or `default` silently treats an explicit `None` as “no override” – backwards from what this problem needs, since `FILECFG`’s `proxy`: `None` must WIN over the default, not defer to it. The imperative fix (a private `_UNSET` sentinel) and the functional fix (`NOTHING`) are the same idea; object-oriented encapsulation doesn’t make the ambiguity go away, it just relocates where you’d have to solve it.

Chapter 14

leaderboard

clamp scores into [0, 100], rank by score desc, name asc

Imperative

```
def rank_imp(rows):
    clamped = []
    for name, score in rows:
        if score < 0:
            score = 0
        elif score > 100:
            score = 100
        clamped.append((name, score))
    clamped.sort(key=lambda r: (-r[1], r[0]))      # score desc, name asc
    return clamped
```

Object-Oriented

```
class Player:
    def __init__(self, name, score):
        self.name = name
        self.score = max(0, min(100, score))      # clamp at construction time

class Leaderboard:
    def __init__(self):
        self.players = []

    def add(self, name, score):
        self.players.append(Player(name, score))
        return self

    def ranked(self):
        ordered = sorted(self.players, key=lambda p: (-p.score, p.name))
        return [(p.name, p.score) for p in ordered]

def rank_oop(rows):
    board = Leaderboard()
    for name, score in rows:
        board.add(name, score)
    return board.ranked()
```

Functional

```
clamp = lambda lo, hi: o(partial(max, lo), partial(min, hi))
clamp100 = clamp(0, 100)
name_of, score_of = itemgetter(0), itemgetter(1)
rank_fn = lambda rows: sortOn(
    lambda r: (-score_of(r), name_of(r)),
    [(name_of(r), clamp100(score_of(r))) for r in rows])
```

Discussion

All three agree, so look at WHERE each one clamps. The imperative version clamps inside the loop with an `if/elif`. The object-oriented version clamps once, in `Player.__init__` – so a `Player` object is a standing proof that its own score is always valid, which is real encapsulation value, not just different syntax for the same check. The functional version clamps by composing two curried `max/min` calls into one reusable function (`clamp`) – no class needed to get the same “validate once, trust everywhere” property, because `clamp100` IS the validated boundary, not an object that happens to carry one.

Chapter 15

logtrriage

'LEVEL message' lines: error count, total, first error message

Imperative

```
def triage_imp(lines):
    errors = 0
    total = 0
    first = None
    seen_first = False
    for ln in lines:
        lvl, _, msg = ln.partition(' ')
        total += 1
        if lvl == 'ERROR':
            errors += 1
            if not seen_first:
                first, seen_first = msg, True
    return ((errors, total), first if seen_first else NOTHING)
```

three mutable slots
and a hand-rolled sentinel:
None alone can't mean "none yet"
if a message could be empty

Functional

```
tag = lambda ln: (lambda lvl, _, msg:
    ((lvl == 'ERROR', 1), msg if lvl == 'ERROR' else NOTHING)
   )(*ln.partition(' '))
triage_fn = partial(foldMap, tag, both(both(Sum, Sum), First))
```

Discussion

Two mutable slots plus a hand-rolled `seen_first` flag in the imperative version, because plain `None` can't distinguish “no error yet” from “the first error had an empty message”. The functional version needs no flag at all: `First`'s identity is `NOTHING`, not `None`, so the fold itself carries the distinction. This is the same bug shape as `layers.py` – `first/seen_first` is a private sentinel invented from scratch, precisely because Python's `None` is overloaded.

Chapter 16

orderbatch

capstone: validate a batch of orders ($\text{qty} > 0$, $\text{price} > 0$), computing each line total; the whole batch fails at the first bad order, carrying which one and why

Imperative

```
def total_imp(order):
    if order["qty"] <= 0:
        return ("err", f"{order['item']}: bad qty {order['qty']}")
    if order["price"] <= 0:
        return ("err", f"{order['item']}: bad price {order['price']}")
    return ("ok", round(order["qty"] * order["price"], 2))

def admit_imp(orders):
    out = []
    for o in orders:
        r = total_imp(o)
        if r[0] == "err":
            return r
        out.append(r[1])
    return ("ok", out)
```

Functional

```
@doE
def total_fn(order):
    qty = yield (Ok(order["qty"]) if order["qty"] > 0
                 else Err(f"{order['item']}: bad qty {order['qty']}"))
    price = yield (Ok(order["price"]) if order["price"] > 0
                  else Err(f"{order['item']}: bad price {order['price']}"))
    return Ok(round(qty * price, 2))

admit_fn = lambda orders: sequenceE([total_fn(o) for o in orders])
```

Discussion

Deliberately no object-oriented version here: an `Order` class with a `.total()` method would just move `total_imp`'s body into a method, changing nothing about how a validation failure PROPAGATES through a whole batch. That propagation – stop at the first bad order, carry which one and why – is what `doE` and `sequenceE` actually buy; encapsulating the per-order check doesn't touch the batch-level control flow, which is the actual hard part of this problem.

Chapter 17

orgchart

resolve an employee’s manager’s manager’s email, or NOTHING if any hop is missing. None-as-Nothing means a missing hop and a real missing-email must stay distinguishable

Imperative

```
def skip_manager_email_imp(emp):
    mgr = MANAGER_OF.get(emp, NOTHING)
    if mgr is NOTHING:
        return NOTHING
    skip = MANAGER_OF.get(mgr, NOTHING)
    if skip is NOTHING:
        return NOTHING
    if skip not in EMAIL_OF:
        return NOTHING
    return EMAIL_OF[skip]
```

Functional

```
@doM
def skip_manager_email_fn(emp):
    mgr = yield maybe_get(MANAGER_OF, emp)
    skip = yield maybe_get(MANAGER_OF, mgr)
    return (yield maybe_get(EMAIL_OF, skip))
```

Discussion

Also deliberately no object-oriented version: wrapping MANAGER_OF/EMAIL_OF in a class would just rename maybe_get/doM as methods, and the real challenge – keeping “employee not found” distinguishable from “employee found, no email on file” – lives in the DATA MODEL, not the calling convention. A hypothetical .skip_manager_email() written with plain dict.get(x) returning None would silently reintroduce the exact ambiguity NOTHING exists to prevent; you would just end up reimplementing a sentinel inside the class anyway.

Chapter 18

payoff

loan payoff schedule: balances after each fixed payment, and how many payments until paid off

Imperative

```
def schedule_imp(balance, payment, rate):
    out, n = [balance], 0
    while balance > 0:
        balance = round(balance * (1 + rate) - payment, 2)
        if balance < 0:
            balance = 0.0
        out.append(balance)
        n += 1
    return out, n
```

Functional

```
def _step(payment, rate):
    def step(balance):
        if balance <= 0:
            return NOTHING
        nxt = round(balance * (1 + rate) - payment, 2)
        nxt = max(nxt, 0.0)
        return (nxt, nxt)
    return step

def schedule_fn(balance, payment, rate):
    rest = unfoldr(_step(payment, rate), balance)
    return [balance] + rest, len(rest)
```

Discussion

The imperative `while` loop and the functional `unfoldr` compute the same recurrence, but `unfoldr`'s stopping condition is DATA (NOTHING) returned from the step function, not a loop condition checked from outside. That distinction matters most for testing: `step(payment, rate)` is a pure function you can call directly with one balance and inspect the answer, with no loop, no mutable balance variable, and no need to run the whole schedule just to check one step's arithmetic.

Chapter 19

sensoralert

smooth a sensor stream with a 3-wide moving average, then report each consecutive run of over-threshold readings as (start, len)

Imperative

```
def alerts_imp(xs, n, threshold):
    smoothed = []
    for i in range(len(xs) - n + 1):
        window = xs[i:i + n]
        smoothed.append(sum(window) / n)
    flags = [v > threshold for v in smoothed]
    runs, i = [], 0
    while i < len(flags):
        if flags[i]:
            start = i
            while i < len(flags) and flags[i]:
                i += 1
            runs.append((start, i - start))
        else:
            i += 1
    return runs
```

Functional

```
def alerts_fn(xs, n, threshold):
    smoothed = [sum(w) / n for w in windows(n, xs)]
    flagged = [(i, v > threshold) for i, v in enum(smoothed)]
    groups = groupBy(lambda a, b: a[1] == b[1], flagged)
    return [(g[0][0], len(g)) for g in groups if g[0][1]]
```

Discussion

`windows` and `groupBy` each replace a hand-rolled index-walking loop with a name. The imperative version's inner `while` (walking flags to find where each run ends) is exactly what `groupBy` does generically – once “runs of the same flag value” is recognised as a `groupBy` problem, the run-finding code disappears entirely, replaced by grouping and then filtering for the runs that are `True`.

Chapter 20

wordfreq

top-N word frequencies, ties alphabetical

Imperative

```
def top_imp(n, text):
    counts = {}                                # mutate a dict
    for w in text.lower().split():
        if w in counts:
            counts[w] += 1
        else:
            counts[w] = 1
    items = list(counts.items())                # then sort, then slice
    items.sort(key=lambda kv: (-kv[1], kv[0]))
    return items[:n]
```

Functional

```
top_fn = lambda n, text: pipe(
    text, str.lower, words,
    lambda ws: fromListWith(add, [(w, 1) for w in ws]),
    dict.items, partial(sortOn, lambda kv: (-kv[1], kv[0])),
    partial(take, n))
```

Discussion

`pipe` reads top to bottom in the order the data actually flows – lowercase, split, count, sort, take – while the imperative version’s five separate steps (build counts, get items, sort in place, slice) are the same five stages with no name tying them together. Neither version is shorter by much; the functional one is easier to extend, because inserting a new stage means inserting one more line in the `pipe` call, not renumbering a sequence of loop variables.

Part III

Appendices

Appendix A

haskell.py, Complete

```
1  """haskell.py -- the comprehensive Haskell-flavoured toolbox for Python.
2
3  One importable vocabulary: the working parts of the Prelude, Data.List,
4  Data.Maybe, Data.Either, Data.Map, Data.Function, Control.Monad, monoids,
5  and parser combinators, folded into sections and mastered as one language.
6
7  Design rules
8  * Where the stdlib already has it, the Haskell name WRAPS the stdlib
9    (scanl is itertools.accumulate, memo is functools.cache) -- one
10   vocabulary, zero reimplementation.
11  * Failure is a value. Maybe uses the true NOTHING sentinel, so None is
12    LEGAL data everywhere; parsers fail with FAIL, and Either carries why.
13  * do-notation via generators: `yield` is >=> (linear monads only).
14  * Constant stack wherever Haskell would lean on TCO: loops, not recursion.
15  * Python keywords and builtins keep prelude.py's underscore convention
16    (not_, map_, filter_, zip_); `id` stays Python's -- use identity.
17  """
18  from functools import reduce as _reduce, partial, cache as memo, cmp_to_key
19  from itertools import accumulate as _acc, islice as _islice, pairwise as _pairwise
20  from itertools import chain, count, cycle, repeat
21  from operator import add, sub, mul, not_
22  from math import gcd, lcm, hypot, isqrt, prod as product
23  from collections import Counter, defaultdict, deque
24  from heapq import heappush, heappop, merge as _hmerge
25  import re as _re
26
27  # ===== 1. functions =====
28  identity = lambda x: x
29  const    = lambda x: lambda _: x
30  flip     = lambda f: lambda a, b: f(b, a)
31  curry    = lambda f: lambda x: lambda y: f(x, y)
32  uncurry  = lambda f: lambda p: f(*p)
33
34  def compose(*fs):
35      """Right-to-left composition, iterative (constant stack).
36      >>> compose(str, abs)(-3)
37      '3'
38      >>> o(str, abs)(-3)
39      '3'
40      """
41      def g(x):
42          for f in reversed(fs):
43              x = f(x)
44          return x
45      return g
46
47  o = compose                # ML's operator: o(f, g) is f after g
48
49  def pipe(x, *fs):
50      """Left-to-right application: pipe(x, f, g) == g(f(x)).
51      >>> pipe(-3, abs, str)
52      '3'
53      """
54      for f in fs:
```

```

55     x = f(x)
56     return x
57
58 def on(f, g):
59     """Data.Function: combine two arguments via a key.
60     >>> on(sub, len)('haskell', 'py')
61     5
62     """
63     return lambda x, y: f(g(x), g(y))
64
65 # ===== 2. pairs =====
66 fst = lambda p: p[0]
67 snd = lambda p: p[1]
68 swap = o(tuple, reversed)
69
70 # ===== 3. sentinels =====
71 class _Sentinel:
72     __slots__ = ("_n",)
73     def __init__(self, n): self._n = n
74     def __repr__(self): return self._n
75     def __bool__(self): return False
76
77 NOTHING = _Sentinel("NOTHING") # Maybe's Nothing; None is a legal Just here
78 FAIL = _Sentinel("FAIL") # parser failure; (None, rest) is a legal success
79 _MISS = _Sentinel("_MISS") # internal: absent optional argument
80
81 # ===== 4. arithmetic & logic =====
82 # add, sub, mul, not_ are operator's; gcd, lcm, hypot, isqrt, product are math's
83 div = lambda a, b: a // b # floors, like Haskell div
84 mod = lambda a, b: a % b # sign follows b
85 halve = lambda n: n // 2
86 even = lambda n: n % 2 == 0
87 odd = lambda n: n % 2 == 1
88 otherwise = True
89
90 def quot(a, b):
91     """Truncating division, Haskell quot; rem's sign follows a.
92     >>> quotRem(-7, 2)
93     (-3, -1)
94     """
95     return -(-a // b) if (a < 0) != (b < 0) else a // b
96 rem = lambda a, b: a - b * quot(a, b)
97 quotRem = lambda a, b: (quot(a, b), rem(a, b))
98 signum = lambda x: (x > 0) - (x < 0)
99 succ = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1
100 pred = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1
101
102 # ===== 5. folds, scans, unfolds =====
103 def foldl(f, xs, base=_MISS):
104     """foldl; seeds with the first element if no base is given.
105     >>> foldl(lambda a, b: a - b, [1, 2, 3], 10)
106     4
107     """
108     return _reduce(f, xs) if base is _MISS else _reduce(f, xs, base)
109
110 def foldr(f, xs, base):
111     """Right fold without recursion.
112     >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
113     [1, 2, 3]
114     """
115     return _reduce(flip(f), reversed(list(xs)), base)
116
117 foldl1 = foldl
118 foldr1 = lambda f, xs: _reduce(flip(f), reversed(list(xs)))
119
120 def scanl(f, xs, base=_MISS):
121     """All intermediate foldl states (itertools.accumulate).
122     >>> scanl(add, [1, 2, 3], 0)
123     [0, 1, 3, 6]
124     """
125     return list(_acc(xs, f) if base is _MISS else _acc(xs, f, initial=base))
126
127 scanl1 = lambda f, xs: scanl(f, xs)
128 scanr = lambda f, xs, base: scanl(flip(f), reversed(list(xs)), base)[::-1]

```

```

129 scanr1 = lambda f, xs: scanl(flip(f), reversed(list(xs)))[:-1]
130
131 def unfoldr(f, seed):
132     """Anamorphism: f(seed) -> (value, seed') or NOTHING to stop.
133     >>> unfoldr(lambda n: NOTHING if n == 0 else (n, n - 1), 4)
134     [4, 3, 2, 1]
135     """
136     out = []
137     while (r := f(seed)) is not NOTHING:
138         out.append(r[0]); seed = r[1]
139     return out
140
141 def until(cond, f, x):
142     """Loop f until cond holds.
143     >>> until(lambda n: n > 100, lambda n: n * 2, 1)
144     128
145     """
146     while not cond(x):
147         x = f(x)
148     return x
149
150 # brute-force enumerations (say the bound out loud):
151 replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]]) # |xs|^n
152 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]]) # 2^n
153
154 # ===== 6. streams ===== (lazy; chain, count, cycle, repeat are itertools')
155 def iterate(f, x):
156     """Infinite stream x, f(x), f(f(x)), ...
157     >>> take(5, iterate(lambda n: n * 2, 1))
158     [1, 2, 4, 8, 16]
159     """
160     while True:
161         yield x
162         x = f(x)
163
164 take = lambda n, xs: list(_islice(iter(xs), n)) # safe on infinite streams
165
166 # ===== 7. list basics =====
167 head = lambda xs: xs[0]
168 tail = lambda xs: xs[1:]
169 init = lambda xs: xs[:-1]
170 last = lambda xs: xs[-1]
171 drop = lambda n, xs: list(xs)[max(n, 0):]
172 splitAt = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)
173 replicate = lambda n, x: [x] * n
174 elem = lambda x, xs: x in xs
175 notElem = lambda x, xs: x not in xs
176 null = lambda xs: len(xs) == 0
177 nub = o(list, dict.fromkeys) # unique, first seen wins
178 enum = lambda xs, start=0: list(enumerate(xs, start))
179 pairwise = lambda xs: list(_pairwise(xs))
180 zip_ = lambda a, b: list(zip(a, b))
181 zip3 = lambda a, b, c: list(zip(a, b, c))
182 unzip = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])
183 isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p)
184 isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p)
185
186 def lookup(k, pairs):
187     """Assoc-list lookup into Maybe (a stored None is data).
188     >>> lookup('b', [('a', 1), ('b', None)]) is None
189     True
190     >>> lookup('c', [('a', 1)]) is NOTHING
191     True
192     """
193     return next((v for kk, v in pairs if kk == k), NOTHING)
194
195 def stripPrefix(p, xs):
196     """The rest after prefix p, or NOTHING.
197     >>> stripPrefix('foo', 'foobar')
198     'bar'
199     >>> stripPrefix('x', 'foobar') is NOTHING
200     True
201     """
202     return xs[len(p):] if isPrefixOf(p, xs) else NOTHING

```

```

203
204 # ===== 8. higher-order lists =====
205 map_      = lambda f, xs: [f(x) for x in xs]
206 filter_   = lambda cond, xs: [x for x in xs if cond(x)]
207 concat    = lambda xss: [x for xs in xss for x in xs]
208 concatMap = lambda f, xs: [y for x in xs for y in f(x)]
209 cross     = lambda a, b: [(x, y) for x in a for y in b]
210 starmap   = lambda f, pairs: [f(*p) for p in pairs]
211 zipWith   = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]
212 zipWith3  = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)]
213
214 def find(cond, xs):
215     """First match, lazily, into Maybe.
216     >>> find(even, [1, 3, 4, 5])
217     4
218     >>> find(even, [1, 3]) is NOTHING
219     True
220     """
221     return next((x for x in xs if cond(x)), NOTHING)
222
223 def partition(cond, xs):
224     """(keepers, rest) in ONE pass, cond called once per element.
225     >>> partition(even, [1, 2, 3, 4])
226     ([2, 4], [1, 3])
227     """
228     yes, no = [], []
229     for x in xs:
230         (yes if cond(x) else no).append(x)
231     return (yes, no)
232
233 # ===== 9. slicing & spans =====
234 def span(cond, xs):
235     """Split at first failure, one pass, one cond call per element.
236     >>> span(lambda x: x < 3, [1, 2, 3, 1])
237     ([1, 2], [3, 1])
238     """
239     xs = list(xs)
240     i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
241     return (xs[:i], xs[i:])
242
243 break_ = lambda cond, xs: span(o(not_, cond), xs)
244
245 def takewhile(cond, xs):                # generator
246     for x in xs:
247         if not cond(x):
248             return
249         yield x
250
251 def dropwhile(cond, xs):                # generator
252     it = iter(xs)
253     for x in it:
254         if not cond(x):
255             yield x
256             break
257     yield from it
258
259 def takeuntil(cond, xs):                # takewhile that INCLUDES the first hit
260     for x in xs:
261         yield x
262         if cond(x):
263             return
264
265 takeWhile = lambda cond, xs: list(takewhile(cond, xs))
266 dropWhile = lambda cond, xs: list(dropwhile(cond, xs))
267 inits     = lambda xs: [xs[:i] for i in range(len(xs) + 1)]
268 tails    = lambda xs: [xs[i:] for i in range(len(xs) + 1)]
269
270 def chunksOf(n, xs):
271     """
272     >>> chunksOf(2, [1, 2, 3, 4, 5])
273     [[1, 2], [3, 4], [5]]
274     """
275     xs = list(xs)
276     return [xs[i:i + n] for i in range(0, len(xs), n)]

```

```

277
278 def windows(n, xs):
279     """Sliding windows of width n.
280     >>> windows(3, [1, 2, 3, 4])
281     [[1, 2, 3], [2, 3, 4]]
282     """
283     xs = list(xs)
284     return [xs[i:i + n] for i in range(len(xs) - n + 1)]
285
286 def groupBy(eq, xs):
287     """Haskell semantics: runs anchored on each run's FIRST element.
288     >>> groupBy(lambda a, b: a == b, "aabba")
289     [['a', 'a'], ['b', 'b'], ['a']]
290     """
291     out = []
292     for x in xs:
293         if out and eq(out[-1][0], x):
294             out[-1].append(x)
295         else:
296             out.append([x])
297     return out
298
299 group = partial(groupBy, lambda a, b: a == b)
300
301 def transpose(rows):
302     """Ragged-safe like Data.List.
303     >>> transpose([[1, 2, 3], [4, 5]])
304     [[1, 4], [2, 5], [3]]
305     """
306     rows = [list(r) for r in rows]
307     out, i = [], 0
308     while any(i < len(r) for r in rows):
309         out.append([r[i] for r in rows if i < len(r)])
310         i += 1
311     return out
312
313 # ===== 10. sorting & searching =====
314 sortOn = lambda f, xs: sorted(xs, key=f)
315 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))
316 maxOn = lambda f, xs: max(xs, key=f)
317 minOn = lambda f, xs: min(xs, key=f)
318 merge = lambda a, b: list(_hmerge(a, b))          # stable merge of sorted lists
319
320 # ===== 11. strings =====
321 lines = str.splitlines
322 unlines = lambda ls: "".join(l + "\n" for l in ls)
323 words = str.split
324 unwords = " ".join
325
326 # ===== 12. containers ===== (Counter, defaultdict, deque, heappush/heappop are stdlib's)
327 def fromListWith(f, pairs):
328     """Data.Map: THE dict-building fold; f(new, old).
329     >>> fromListWith(add, [('a', 1), ('b', 2), ('a', 3)]) == {'a': 4, 'b': 2}
330     True
331     """
332     d = {}
333     for k, v in pairs:
334         d[k] = f(v, d[k]) if k in d else v
335     return d
336
337 def insertWith(f, k, v, d):
338     """Persistent insert (fresh dict); f(new, old).
339     >>> insertWith(add, 'a', 5, {'a': 1}) == {'a': 6}
340     True
341     """
342     return {**d, k: f(v, d[k]) if k in d else v}
343
344 def unionWith(f, a, b):
345     """Data.Map: merge two dicts, combining shared keys with f.
346     >>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4}) == {'a': 1, 'b': 5, 'c': 4}
347     True
348     """
349     return {k: f(a[k], b[k]) if k in a and k in b else (a[k] if k in a else b[k])
350             for k in a.keys() | b.keys()}

```

```

351
352 def getpath(t, ks, default=NOTHING):
353     """Read along a key path without autovivifying; Maybe out.
354     >>> getpath({'a': {'b': None}}, ['a', 'b']) is None
355     True
356     >>> getpath({'a': {}}, ['a', 'b']) is NOTHING
357     True
358     """
359     for k in ks:
360         if not isinstance(t, dict) or k not in t:
361             return default
362         t = t[k]
363     return t
364
365 # multiset ops on count-dicts (use .get -- indexing a Counter autovivifies):
366 bag_diff = lambda a, b: {k: a[k] - b.get(k, 0) for k in a if a[k] > b.get(k, 0)}
367 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a[k], b[k]))}
368 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
369 bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())
370
371 # ===== 13. Maybe ===== (NOTHING sentinel; None is a legal value)
372 def bind(x, f):
373     """>>= : short-circuit on NOTHING.
374     >>> bind(3, lambda v: v + 1), bind(NOTHING, lambda v: v + 1)
375     (4, NOTHING)
376     """
377     return NOTHING if x is NOTHING else f(x)
378
379 isJust = lambda x: x is not NOTHING
380 isNothing = lambda x: x is NOTHING
381 fromMaybe = lambda default, x: default if x is NOTHING else x
382
383 listToMaybe = lambda xs: xs[0] if xs else NOTHING
384 maybeToList = lambda x: [] if x is NOTHING else [x]
385
386 def catMaybes(xs):
387     """
388     >>> catMaybes([1, NOTHING, None, 2])
389     [1, None, 2]
390     """
391     return [x for x in xs if x is not NOTHING]
392
393 def mapMaybe(f, xs):
394     """Map into Maybe and drop the NOTHINGS, one pass.
395     >>> mapMaybe(lambda s: int(s) if s.isdigit() else NOTHING, ['3', 'x', '7'])
396     [3, 7]
397     """
398     return [y for y in map(f, xs) if y is not NOTHING]
399
400 def sequenceM(xs):
401     """All-or-nothing.
402     >>> sequenceM([1, 2]), sequenceM([1, NOTHING])
403     ([1, 2], NOTHING)
404     """
405     out = []
406     for x in xs:
407         if x is NOTHING:
408             return NOTHING
409         out.append(x)
410     return out
411
412 traverseM = lambda f, xs: sequenceM(map_(f, xs))
413
414 def maybe_get(d, k):
415     """dict lookup into Maybe (None stays distinguishable from missing).
416     >>> maybe_get({'a': None}, 'a'), maybe_get({}, 'a')
417     (None, NOTHING)
418     """
419     return d.get(k, NOTHING)
420
421 # ===== 14. Either ===== (tagged tuples; match-friendly)
422 Ok = lambda v: ("ok", v)
423 Err = lambda e: ("err", e)
424

```

```

425 def bindE(x, f):
426     """
427     >>> bindE(Ok(3), lambda v: Ok(v + 1)), bindE(Err("no"), lambda v: Ok(v))
428     (('ok', 4), ('err', 'no'))
429     """
430     return x if x[0] == "err" else f(x[1])
431
432 def sequenceE(xs):
433     """First Err wins; else Ok of all values.
434     >>> sequenceE([Ok(1), Err("a"), Err("b")])
435     ('err', 'a')
436     """
437     out = []
438     for x in xs:
439         if x[0] == "err":
440             return x
441         out.append(x[1])
442     return Ok(out)
443
444 def traverseE(f, xs):
445     """Map into Either, then sequence.
446     >>> traverseE(lambda n: Ok(n * 2) if n else Err('zero'), [1, 2])
447     ('ok', [2, 4])
448     """
449     return sequenceE(map_(f, xs))
450
451 def partitionEithers(xs):
452     """
453     >>> partitionEithers([Ok(1), Err('a'), Ok(2)])
454     ([1, 2], ['a'])
455     """
456     oks, errs = [], []
457     for tag, v in xs:
458         (oks if tag == "ok" else errs).append(v)
459     return (oks, errs)
460
461 def note(msg, x):
462     """Maybe -> Either: attach WHY to a NOTHING.
463     >>> note("missing key", NOTHING)
464     ('err', 'missing key')
465     """
466     return Err(msg) if x is NOTHING else Ok(x)
467
468 # ===== 15. do-notation ===== (yield is >>=; linear monads only)
469 def do(binder, is_fail):
470     """Build a do-block decorator for any short-circuiting monad."""
471     def deco(gen_fn):
472         def run(*a, **kw):
473             g = gen_fn(*a, **kw)
474             def step(val):
475                 try:
476                     m = g.send(val)
477                     except StopIteration as e:
478                         return e.value
479                     return binder(m, step)
480             return step(None)
481         return run
482     return deco
483
484 doM = do(bind, lambda x: x is NOTHING) # generator returns a plain value
485 doE = do(bindE, lambda x: x[0] == "err") # generator returns Ok(...)/Err(...)
486
487 # ===== 16. parser combinators =====
488 # Parser a = str -> (value, rest) | FAIL. Values may be None (json null!).
489 def doP(gen_fn):
490     """do-notation that THREADS the input: yield a parser, receive its value.
491     Fresh generator per parse -- re-runnable, backtrack-safe under alt."""
492     def make(*a, **kw):
493         def parser(s):
494             g, val = gen_fn(*a, **kw), None
495             try:
496                 while True:
497                     r = g.send(val)(s)
498                     if r is FAIL:

```

```

499         return FAIL
500         val, s = r
501     except StopIteration as e:
502         return (e.value, s)
503     return parser
504 return make
505
506 def alt(*ps):
507     """</> : first success wins."""
508     return lambda s: next((r for r in (p(s) for p in ps) if r is not FAIL), FAIL)
509
510 def many(p):
511     """Zero or more, loop not recursion; stalls guard against empty matches."""
512     def parser(s):
513         out = []
514         while (r := p(s)) is not FAIL and r[1] != s:
515             out.append(r[0]); s = r[1]
516         return (out, s)
517     return parser
518
519 def many1(p):
520     def parser(s):
521         r = p(s)
522         if r is FAIL:
523             return FAIL
524         rest = many(p)(r[1])
525         return ([r[0]] + rest[0], rest[1])
526     return parser
527
528 def sepBy(p, sep):
529     def pair(s):
530         r = sep(s)
531         return FAIL if r is FAIL else p(r[1])
532     def parser(s):
533         r = p(s)
534         if r is FAIL:
535             return ([], s)
536         rest = many(pair)(r[1])
537         return ([r[0]] + rest[0], rest[1])
538     return parser
539
540 def lit(c):
541     """One expected character, whitespace-blind."""
542     def parser(s):
543         s = s.lstrip()
544         return (c, s[1:]) if s[:1] == c else FAIL
545     return parser
546
547 def rx(pat, conv=identity):
548     """Regex token, whitespace-blind, converted."""
549     r = _re.compile(pat)
550     def parser(s):
551         s = s.lstrip()
552         m = r.match(s)
553         return (conv(m.group()), s[m.end():]) if m else FAIL
554     return parser
555
556 def chainl1(p, ops):
557     """p (op p)* folded LEFT; ops maps operator char -> binary function."""
558     opp = alt(*[lit(c) for c in ops])
559     def parser(s):
560         vr = p(s)
561         while vr is not FAIL:
562             op = opp(vr[1])
563             if op is FAIL:
564                 return vr
565             w = p(op[1])
566             if w is FAIL:
567                 return vr
568             vr = (ops[op[0]](vr[0], w[0]), w[1])
569         return FAIL
570     return parser
571
572 def runParser(p, s):

```

```

573     """Parser -> Either: Ok(value) or Err showing where it stopped.
574     >>> runParser(rx(r'\d+', int), "42")
575     ('ok', 42)
576     >>> runParser(rx(r'\d+', int), "42x")
577     ('err', "unconsumed input: 'x'")
578     """
579     r = p(s)
580     if r is FAIL:
581         return Err(f"no parse: {s!r}")
582     if r[1].strip():
583         return Err(f"unconsumed input: {r[1].strip()!r}")
584     return Ok(r[0])
585
586 # ===== 17. monoids =====
587 Monoid = lambda empty, op: (empty, op)
588 mconcat = lambda m, xs: foldl(m[1], xs, m[0])
589 foldMap = lambda f, m, xs: mconcat(m, [f(x) for x in xs])
590 both = lambda m1, m2: Monoid((m1[0], m2[0]),
591                               lambda a, b: (m1[1](a[0], b[0]), m2[1](a[1], b[1])))
592 Sum = Monoid(0, add)
593 Product = Monoid(1, mul)
594 All = Monoid(True, lambda a, b: a and b)
595 Any = Monoid(False, lambda a, b: a or b)
596 MaxM = Monoid(float("-inf"), max)
597 MinM = Monoid(float("inf"), min)
598 ListM = Monoid([], add)
599 First = Monoid(NOTHING, lambda a, b: b if a is NOTHING else a)
600 Last = Monoid(NOTHING, lambda a, b: a if b is NOTHING else b)
601
602 if __name__ == "__main__":
603     import doctest
604     fails, total = doctest.testmod()
605     print(f"{total - fails}/{total} doctests passed" if not fails
606           else f"{fails} FAILED")

```

